

Duc Nguyen - UCID: 30102065

Lab section B01

ENCM 335 - LAB 7

Exercise B

Code:

main7B.c

```
1 // main7B.c
2 // ENCM 335 Fall 2021 Lab 7 Exercise B
3
4 // main function to demonstrate operations on vectors in 3-space.
5
6 #include <stdio.h>
7 #include "vec7B.h"
8
9 int main(void)
10 {
11     vec_t u = { 1.5, 0.5, 0.125 }, v = { -2.0, 3.5, -3.0 };
12
13     printf("Value of u: ");
14     print_vec(u);
15     printf("\nValue of v: ");
16     print_vec(v);
17     printf("\n\n");
18
19     // Demonstrate scalar product.
20     double k = 0.5;
21     vec_t sp;
22     scalar_product(k, &v, &sp);
23     printf("Scalar product of %g and v is: ", k);
24     print_vec(sp);
25     printf("\n\n");
26
27     printf("The dot product of u and v is: %g \n\n", dot_product(u, v));
28
29     vec_t sum_vector;
30     printf("The sum of u and v is: ");
31
32     sum(&u, &v, &sum_vector);
33     print_vec(sum_vector);
34     printf("\n\n");
35
36     printf("The cross product of u and v is: ");
37     print_vec(cross_product(u, v));
38     return 0;
39 }
```

vec7B.c

```
1 // vec7B.c
2 // ENCM 335 Fall 2021 Lab 7 Exercise B
3
4 // Function definitions for operations on vectors in 3-space.
5
6 #include <stdio.h>
7 #include "vec7B.h"
8
9 void print_vec(vec_t v)
10 {
11     // REMARK: %g usually does a more helpful job of printing a double
12     // than %f does.
13     printf("(%g %g %g)", v.x, v.y, v.z);
14 }
15
16 void scalar_product(double k, const vec_t *v, vec_t *w)
17 {
18     w->x = k * v->x;
19     w->y = k * v->y;
20     w->z = k * v->z;
21 }
22
23 double dot_product(vec_t left, vec_t right) {
24     double result;
25     result = left.x*right.x + left.y*right.y + left.z*right.z;
26     return result;
27 }
28
29 void sum(const vec_t *left, const vec_t *right, vec_t *sum) {
30     sum->x = left->x + right->x;
```

```

31     sum->y = left->y+right->y;
32     sum->z = left->z+right->z;
33 }
34
35 vec_t cross_product(vec_t left, vec_t right){
36     vec_t result;
37     result.x=left.y*right.z-left.z*right.y;
38     result.y=left.z*right.x-left.x*right.z;
39     result.z=left.x*right.y-left.y*right.x;
40     return result;
41 }

```

vec7B.h

```

1 // vec7B.h
2 // ENCM 335 Fall 2021 Lab 7 Exercise B
3
4 // Type information and function prototypes for operations on
5 // vectors in 3-space.
6
7 struct vec {
8     double x;
9     double y;
10    double z;
11 };
12
13 typedef struct vec vec_t;
14
15 void print_vec(vec_t v);
16 // PROMISES:
17 //   Members of v are printed in a format as in this example:
18 //   (-1.25 0.5 19.0)
19
20
21 // REMARK: The interfaces below do not use a consistent style for
22 //   communication of inputs and results of functions. That would be
23 //   annoying in production code, but in this exercise it helps
24 //   students learn when to use the . operator and when to use the ->
25 //   operator.
26
27 void scalar_product(double k, const vec_t *v, vec_t *w);
28 // REQUIRES: v and w point to vector objects.
29 // PROMISES:
30 //   *w contains the scalar product of k and *v.
31
32
33 double dot_product(vec_t left, vec_t right);
34 // PROMISES:
35 //   Return value is dot product of left and right.
36
37 void sum(const vec_t* left, const vec_t *right, vec_t *sum);
38 // REQUIRES:
39 //   left, right and sum point to vector objects.
40 // PROMISES:
41 //   *sum contains the vector sum of *left and *right.
42
43 vec_t cross_product(vec_t left, vec_t right);
44 // PROMISES:
45 //   Return value is cross product of left and right.

```

Output:

```

monsi@LAPTOP-M9ONH6KV ~/ENCM_335/lab7
$ ./a.exe
Value of u: (1.5 0.5 0.125)
Value of v: (-2 3.5 -3)

Scalar product of 0.5 and v is: (-1 1.75 -1.5)

The dot product of u and v is: -1.625

The sum of u and v is: (-0.5 4 -2.875)

The cross product of u and v is: (-1.9375 4.25 6.25)
monsi@LAPTOP-M9ONH6KV ~/ENCM_335/lab7

```

Exercise C

Code:

```
1 // grades7C.c
2 // ENCM 335 Fall 2021 Lab 7 Exercise C
3
4 #include <stdio.h>
5 #include <string.h>
6
7 #define NUM_OF_ASSIGNMENTS 10
8
9 struct mark_record {
10     char last_name[8]; // way too short for practical use!
11     char initials[4];
12     int id_number;
13     double mark[NUM_OF_ASSIGNMENTS];
14 };
15
16 typedef struct mark_record mark_record_t;
17
18 void list_students(const mark_record_t *record, int n);
19 // REQUIRES:
20 //   n >= 1.
21 //   Elements record[0] ... record[n-1] exist and have valid
22 //   data in their last name, initials, and id_number members.
23 // PROMISES:
24 //   Students' names and ID numbers are printed, one line per student,
25 //   in this format:
26 //   last_name, initials    id_number
27
28 void list_assignment_totals(const mark_record_t *record,
29                             int n,
30                             const double *maxes);
31
32 // REQUIRES:
33 //   n >= 1.
34 //   Elements record[0] ... record[n-1] exist, have valid
35 //   data in their last name, initials, and id_number members,
36 //   and valid marks in their marks arrays.
37 // PROMISES:
38 //   Students' names, ID numbers, and overall assignment scores
39 //   are printed, one per line, in the format given in the Lab 5
40 //   Exercise D instructions.
41
42 int main(void)
43 {
44     // It would make sense to read scores and maximum score data from
45     // an input file, but we don't know how to do that yet in ENCM 335
46     // this term. So we'll just put test data directly into a local
47     // variable in main.
48     mark_record_t test_scores[] = {
49         {
50             "Cross", "JP", 900431,
51             {8.5, 8.0, 11.0, 8.5, 8.0, 11.0, 8.5, 7.0, 11.5, 7.5}
52         },
53         {
54             "Dodd", "AJ", 900872,
55             {8.5, 8.5, 10.5, 9.0, -1.0, 9.5, 9.5, 7.0, 12.0, 5.0}
56         },
57         {
58             "Moss", "WLM", 901203,
59             {7.5, 10.0, 12.0, 8.5, 7.0, 11.0, 10.0, -1.0, 13.0, -1.0}
```

```

59 | },
60 | {
61 |     "Ross", "T", 900398,
62 |     {7.5, -1.0, 11.0, 8.5, 7.0, 12.0, 9.0, -1.0, 12.5, 8.0}
63 | },
64 | {
65 |     "Todd", "BL", 901197,
66 |     {9.0, 8.0, -1.0, 6.0, 8.0, 9.5, 11.0, 10.0, -1.0, 6.0}
67 | }
68 | };
69 |
70 | // Change the next line to #if 1 as part of solving the problem.
71 | #if 1
72 | double max_scores[ ] = {
73 |     10.0, 10.0, 12.0, 9.0, 10.0, 12.0, 11.0, 10.0, 13.0, 8.0
74 | };
75 | #endif
76 |
77 | // Replace this call with a call to list_assignment_totals.
78 | list_assignment_totals(test_scores, sizeof(test_scores) / sizeof(mark_record_t), max_scores);
79 |
80 | return 0;
81 | }
82 |
83 | void list_students(const mark_record_t *record, int n)
84 | {
85 |     int i;
86 |     char name[20];
87 |
88 |     for (i = 0; i < n; i++) {
89 |         strcpy(name, record[i].last_name);
90 |         strcat(name, ", ");
91 |         strcat(name, record[i].initials);
92 |         printf("%-15s %6d\n", name, record[i].id_number);
93 |     }
94 |
95 |     // Insert a definition for list_assignment_totals here.
96 |     void list_assignment_totals(const mark_record_t *record, int n, const double *maxes) {
97 |         int i, j;
98 |         double total_weights, total_marks;
99 |         char name[20];
100 |         for (i=0; i<n; i++){
101 |             total_weights=0.0;
102 |             total_marks=0.0;
103 |
104 |             strcpy(name, record[i].last_name);
105 |             strcat(name, ", ");
106 |             strcat(name, record[i].initials);
107 |
108 |             for (j=0; j<NUM_OF_ASSIGNMENTS; j++){
109 |                 if (record[i].mark[j] != -1.0){
110 |                     total_weights += maxes[j];
111 |                     total_marks += record[i].mark[j];
112 |                 }
113 |             }
114 |             //print out the formatted data
115 |             printf("%-15s %6d    %5.1f / %5.1f, %.2f%%\n", name, record[i].id_number,
116 |                 total_marks, total_weights, total_marks/total_weights*100);
117 |         }
118 |     }

```

Output:

```

monsi@LAPTOP-M9ONH6KV ~/ENCM_335/lab7
$ ./a.exe
Cross, JP          900431      89.5 / 105.0, 85.24%
Dodd, AJ           900872      79.5 / 95.0, 83.68%
Moss, WLM          901203      79.0 / 87.0, 90.80%
Ross, T            900398      75.5 / 85.0, 88.82%
Todd, BL           901197      67.5 / 80.0, 84.38%

```

Exercise E

